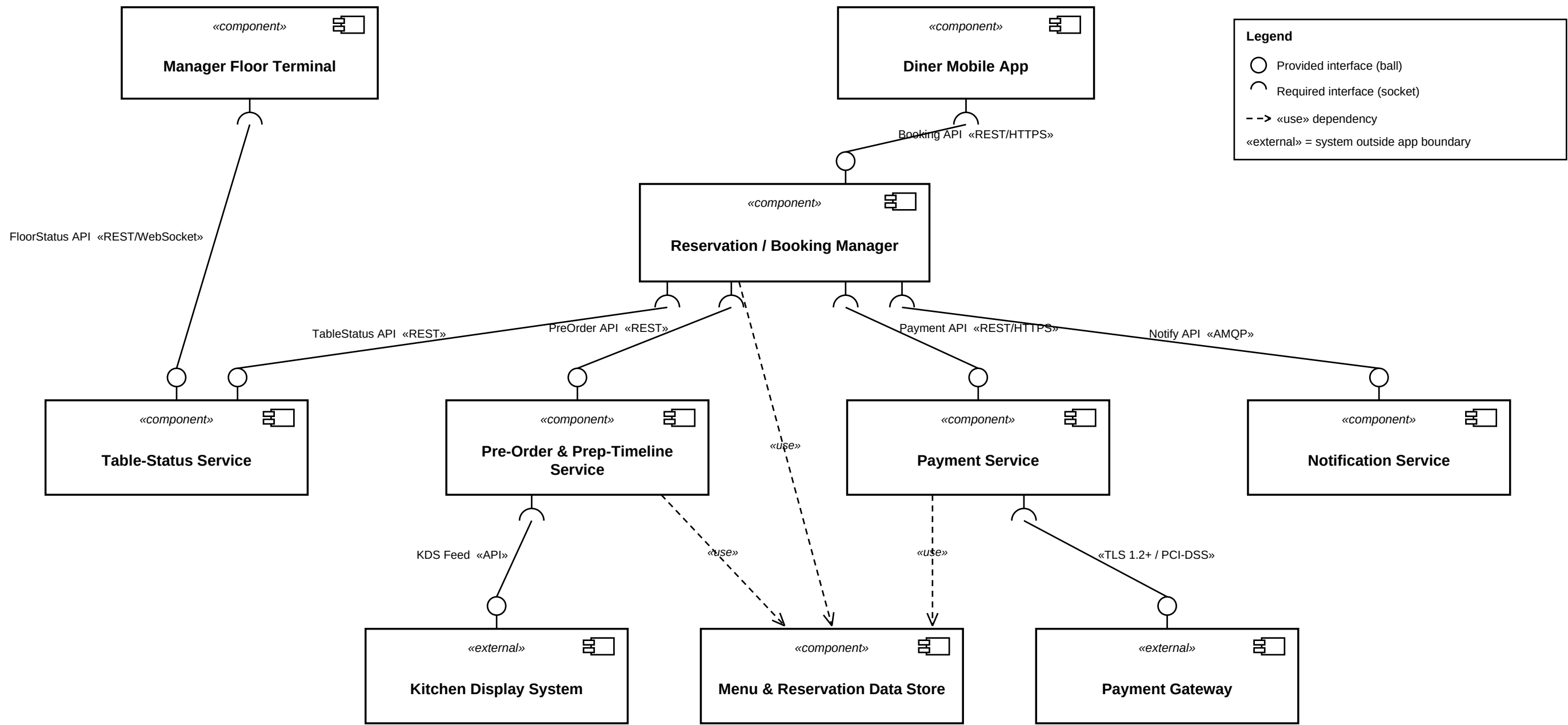


Restaurant Table Booking & Pre-Ordering App — Component Diagram (Microservices)

PS #36 · Lab 3: Component Modelling & Architectural Pattern Selection



Lab 3 — Architectural Justification

PS #36 : Restaurant Table Booking & Pre-Ordering App

Architecture Selection: We chose the Microservices Architecture for the Restaurant Table Booking & Pre-Ordering System.

Architectural Choice

The system is decomposed into small, independently deployable services — Reservation/Booking Manager, Table-Status Service, Pre-Order & Prep-Timeline Service, Payment Service and Notification Service — each owning a single capability and communicating through well-defined provided/required interfaces. Two systems, the Payment Gateway and the Kitchen Display System, sit outside the application boundary and are reached only through their owning services.

Two Reasons for the Choice

Reason 1 — Independent scaling for uneven load. Booking and table-status traffic spike sharply during peak dining windows, while payment and notification volumes follow a different curve. A microservices design lets each service scale horizontally on its own, so the app can absorb a dinner rush without over-provisioning the entire stack — directly supporting the peak-load behaviour required by NFR-001.

Reason 2 — Fault isolation across external dependencies. The app depends on an external Payment Gateway and Kitchen Display System. Here a gateway timeout or a KDS outage is contained inside the Payment or Pre-Order service; a diner can still view the floor plan and hold a table — mirroring alternate flow 7a, where a declined payment keeps the temporary hold active — rather than the whole system going down with one dependency.

Security Advantage

Isolating all card handling inside a single Payment Service shrinks the PCI-DSS compliance scope to one hardened, separately deployed boundary. Only that service transmits card data to the gateway over TLS 1.2+, so the remaining services never touch cardholder data. This satisfies NFR-002 while keeping the security audit surface small and easy to reason about.

Performance Benefit

The dedicated Table-Status Service can push status changes to every manager floor terminal through a lightweight publish/subscribe channel (e.g. WebSocket) backed by an in-memory cache, instead of having each terminal poll a shared database. This keeps the 95th-percentile sync latency under the 500 ms target set by NFR-001 even under simulated peak load, and lets the real-time status path scale independently of the heavier booking and payment paths.